Architecture Decisions for the Production-Grade Retrieval Platform

1. Purpose and Scope

This system is a modular hybrid retrieval platform designed to ingest heterogeneous documents, process them into searchable units, and provide high-quality search through a unified API. It supports PDF, CSV, TXT, and Markdown-style inputs, and combines lexical and semantic retrieval to improve robustness across query types.

The architecture prioritises three core goals: **retrieval quality**, **modularity**, and **low operational overhead**. This document outlines the key design decisions, alternatives considered, and the trade-offs inherent in the chosen approach.

2. System Requirements and Constraints

Functional requirements

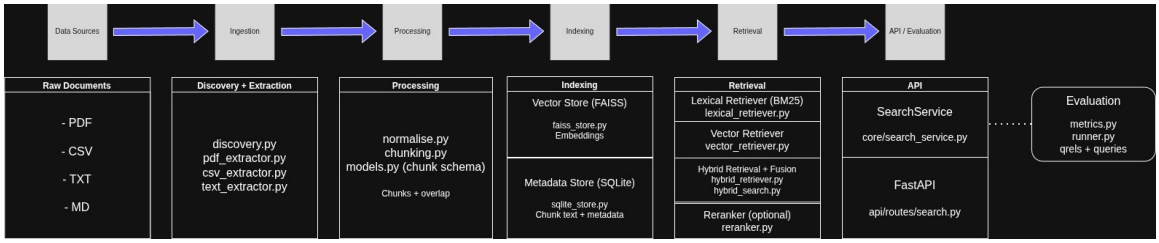
- Ingest and process multiple file formats
- Support semantic, lexical, and hybrid retrieval
- Provide a search API and CLI interface
- Enable offline evaluation using standard IR metrics

Non-functional requirements

- Locally runnable with minimal setup
- Low-latency retrieval for interactive use
- Modular and extensible architecture
- Testable and reproducible pipeline

The system was intentionally scoped as a **single-node, locally deployable system** while still reflecting production-oriented design principles.

3. High-Level Architecture



The system is structured into distinct layers:

- **Ingestion Layer**: Discovers files and extracts content using format-specific extractors (`extractors/`)
- **Processing Layer**: Normalises and chunks text (`processing/`)
- **Indexing Layer**:
  - FAISS for vector similarity search (`faiss_store.py`)
  - SQLite for metadata persistence (`sqlite_store.py`)
- **Retrieval Layer**:
  - Lexical (BM25-style) retrieval
  - Vector (embedding-based) retrieval
  - Hybrid fusion and optional reranking (`retrieval/`)
- **Service Layer**: Orchestrated via `SearchService` (`core/search_service.py`)
- **API Layer**: Exposed through FastAPI (`api/routes/search.py`)
- **Evaluation Layer**: Offline benchmarking (`evaluation/`)

4. Key Architecture Decisions

4.1 Hybrid Retrieval (Lexical + Semantic)

Decision: Combine BM25-style lexical retrieval with vector-based semantic retrieval.

Rationale: Lexical retrieval performs well for exact keyword matches, while semantic retrieval captures meaning and paraphrases. Combining both improves recall and robustness across diverse queries.

Alternatives rejected:

- Lexical-only (poor semantic understanding)
- Semantic-only (misses exact matches)

Trade-offs:

- Increased system complexity
- Higher query latency due to dual retrieval

4.2 Chunk-Based Indexing

Decision: Index documents as overlapping chunks instead of full documents.

Rationale: Improves retrieval granularity and relevance for long documents, especially technical PDFs.

Alternatives rejected:

- Full-document embeddings (too coarse)
- Sentence-level chunking (too fragmented)

Trade-offs:

- Increased storage and metadata management
- Requires tuning chunk size and overlap

4.3 FAISS for Vector Storage

Decision: Use FAISS for dense vector indexing.

Rationale: Provides fast approximate nearest neighbour search with no external dependencies, making it ideal for local deployment.

Alternatives rejected:

- Managed vector databases (e.g., Pinecone) due to added cost and complexity
- PostgreSQL + pgvector due to lower performance for this use case

Trade-offs:

- Limited filtering capabilities
- File-based persistence instead of transactional guarantees

4.4 SQLite for Metadata Storage

Decision: Store chunk metadata and mappings in SQLite.

Rationale: Lightweight, embedded, and sufficient for structured metadata without requiring external infrastructure.

Alternatives rejected:

- PostgreSQL (heavier setup)
- MongoDB (unnecessary flexibility)

Trade-offs:

- Limited concurrency and scalability
- Not suitable for distributed systems

4.5 Modular Ingestion Pipeline

Decision: Separate ingestion into discovery, extraction, processing, and indexing stages.

Rationale: Improves maintainability, testability, and extensibility. Each stage can evolve independently.

Alternatives rejected:

- Monolithic ingestion pipeline

Trade-offs:

- Increased number of modules

- Slight coordination overhead between components

4.6 Format-Specific Extractors (Strategy Pattern)

Decision: Implement separate extractors for each file type behind a shared interface.

Rationale: Each file format has distinct parsing requirements; separation avoids complexity and improves extensibility.

Alternatives rejected:

- Single generic extractor with branching logic

Trade-offs:

- Some duplication across extractors
- Easier long-term maintenance

4.7 Centralised Retrieval Orchestration (SearchService)

Decision: Use a dedicated service layer to coordinate retrieval logic.

Rationale: Keeps API thin and separates orchestration (fusion, reranking) from retrieval components.

Alternatives rejected:

- Embedding logic inside routes
- Distributing orchestration across retrievers

Trade-offs:

- Central service becomes a critical component
- Requires discipline to avoid overloading responsibilities

4.8 Optional Reranking Layer

Decision: Implement reranking as a separate, optional stage.

Rationale: Improves ranking quality (e.g., nDCG) while allowing flexible performance configurations.

Alternatives rejected:

- No reranker (lower precision)
- Always-on heavy reranker (higher latency)

Trade-offs:

- Additional latency when enabled
- Additional model dependency

4.9 Built-in Evaluation Framework

Decision: Include offline evaluation with qrels and IR metrics.

Rationale: Enables measurable correctness and comparison between retrieval strategies.

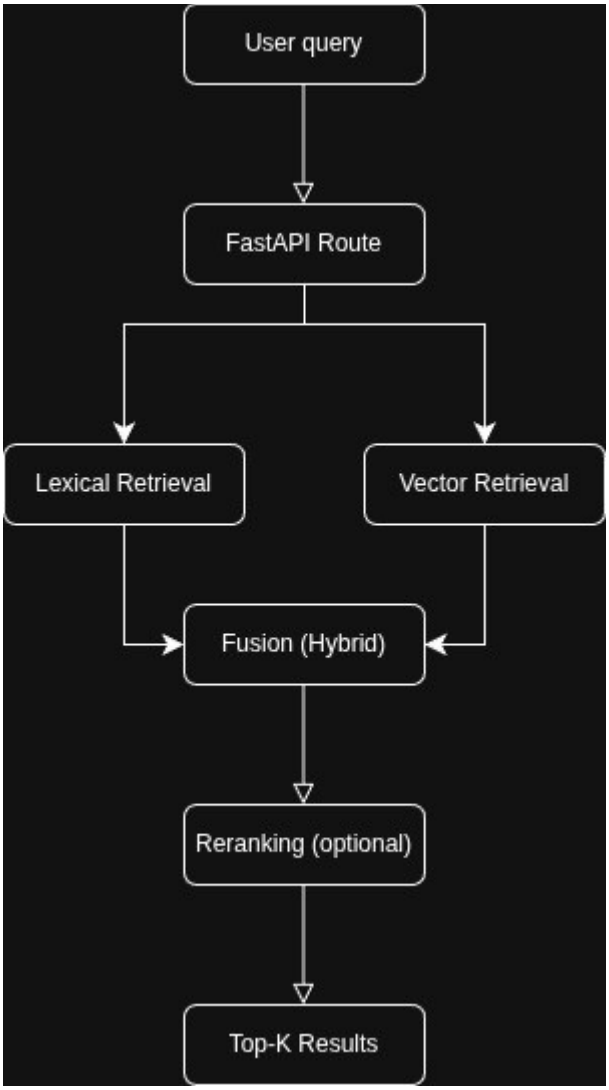
Alternatives rejected:

- Manual testing only

Trade-offs:

- Requires relevance labeling
- Additional setup effort

5. Query Flow



At query time:

1. Request enters FastAPI endpoint
2. Routed to `SearchService`
3. Query processed by:
    - Lexical retriever
    - Vector retriever
4. Results fused and optionally reranked
5. Top-k results returned

6. Trade-offs and Production Considerations

- **Scalability:** SQLite and local FAISS limit horizontal scaling; suitable for single-node deployment
- **Latency:** Hybrid retrieval and reranking increase query cost but improve relevance
- **Reliability:** File-based persistence is simple but requires careful management
- **Extensibility:** Strong modular design enables easy addition of new extractors, retrievers, and models
- **Operational complexity:** Minimal, due to local-first design

7. Future Evolution

With additional time, the system could evolve by:

- Replacing SQLite with PostgreSQL for concurrency
- Moving FAISS to a distributed or managed vector store
- Adding monitoring and observability (latency, success rates)
- Supporting asynchronous ingestion pipelines
- Introducing metadata filtering and faceted search

8. Conclusion

The final architecture prioritises modularity, measurable correctness, and low operational overhead while maintaining a clear path toward production scalability. The combination of FastAPI, FAISS, SQLite, and a hybrid retrieval design provides a strong balance between performance, simplicity, and extensibility.